Islamic University of Technology (IUT)

Department of Electrical and Electronic Engineering

Project Report

Design and Implementation of a Low-Cost 8051-Based Digital Alarm Clock with Multi-Alarm, Snooze, and Secure Stop-Sequence Features on the AT89C51 Microcontroller Platform

Course No.	EEE 4706
Course Name	Microcontroller Based System Design Lab
Academic Year / Semester	2024-25 / Winter
Course Instructor	Prof. Nadim Ahmed
Project Team	C2 Team 1
Submission Date	April 19, 2026

Group: C2

Team no: 1

Team members:

Student name	Student ID
Ar-Rafi Ishraq	210021330
Md. Iftinum Ul Haque	210021326
Miah Raahnan Wahid Purna	210021334
Sharia Hamza	210021354
Fakri Issihaka	210021352

1. Abstract

This report presents the design, implementation, and evaluation of a low-cost digital alarm clock built around the Atmel AT89C51 (8051-family) microcontroller. The system provides accurate 24-hour timekeeping through a hardware-timer based real-time tick, supports three independent user-configurable alarms, offers a configurable snooze function, and introduces a novel multi-key stop sequence (1-2-1) designed to prevent accidental alarm dismissal. User interaction is delivered via a 16×2 HD44780 LCD and a 4×4 matrix keypad, while audible and visual alerts are generated using an active-low buzzer and a pair of status LEDs. The entire firmware is written in hand-optimised 8051 assembly (MIDE-51 syntax), using a 12 MHz crystal oscillator and a Timer 0 interrupt-driven 10 ms tick. The completed system demonstrates that robust, feature-rich embedded products can be realised on a minimal component budget while still meeting reliability and usability targets.

2. Introduction

Time-keeping and scheduled alerts are among the most ubiquitous applications of digital electronics in daily life. Although commercial alarm clocks are inexpensive, designing one from first principles on a general-purpose microcontroller remains a canonical exercise in embedded systems education. The process exposes students to a complete vertical slice of the discipline: low-level register configuration, interrupt handling, real-time software design, human-machine interface (HMI) development, peripheral driving, and end-to-end system integration.

This project was undertaken as the Complex Engineering Problem (CEP) for the course Microcontroller Based System Design (EEE 4705). The AT89C51 was selected as the target platform because it is architecturally representative of the classic 8051 family, widely available, economical, and well supported by educational tooling such as the Proteus VSM simulator and the ASEM-51 assembler.

Beyond merely re-implementing a conventional clock, this design incorporates two pedagogically valuable extensions: (i) a synchronised LED flash tied to the alarm buzzer, providing redundant visual indication, and (ii) a "1→2→1" key-sequence stop mechanism that forces the user to perform a deliberate action before the alarm is silenced. The latter addresses a real usability pitfall of conventional alarm clocks, where a single button press in a drowsy state often leads to oversleeping.

3. Project Objectives

The specific objectives of the project are summarised below:

- To design a functional 24-hour digital clock on the AT89C51 microcontroller using a hardware timer interrupt for accurate 1-second tick generation.
- To implement a 16×2 character LCD-based user interface for displaying current time, alarm status, and menu prompts.
- To enable the user to set the current time and configure up to three independent alarms via a 4×4 matrix keypad.
- To provide an audible alert through an active-low buzzer and a synchronised visual indication through an LED.
- To implement a configurable snooze facility allowing the user to defer an active alarm by a chosen number of minutes.
- To implement a secure multi-key stop sequence (1→2→1) that prevents accidental or subconscious alarm dismissal.
- To write the entire firmware in optimised 8051 assembly language and verify correct operation using the Proteus VSM simulator.

4. System Architecture and Block Diagram

The system architecture, illustrated conceptually below, is organised around the AT89C51 microcontroller acting as the central processing and control unit. Four peripheral subsystems are connected to its general-purpose I/O ports:

- A 16×2 character LCD (LM016L, HD44780-compatible) connected in 8-bit parallel mode for alphanumeric display.
- A 4×4 matrix keypad for user input, scanned using row-drive/column-read multiplexing.
- A piezoelectric buzzer driven in an active-low configuration for the audible alarm.
- Two status LEDs (red and green) driven through current-limiting resistors for visual indication.

A 12 MHz crystal oscillator provides the master clock, while a standard reset network (10 μF capacitor in series with a 10 kΩ resistor) ensures a reliable power-on reset. The EA pin is tied to V_{ee} so that firmware execution begins from the internal Flash memory.

Logically, the firmware is partitioned into three concurrent layers: a foreground main loop (which drives the LCD and scans for user input), a Timer 0 interrupt service routine (which maintains the real-time counters), and a collection of blocking subroutines invoked in response to events such as alarm triggers or menu selections.

5. Hardware Design and Schematic

The complete hardware schematic of the digital alarm clock, as designed and verified in the Proteus VSM environment, is shown in Figure 1. Each functional block is described in the subsections that follow.

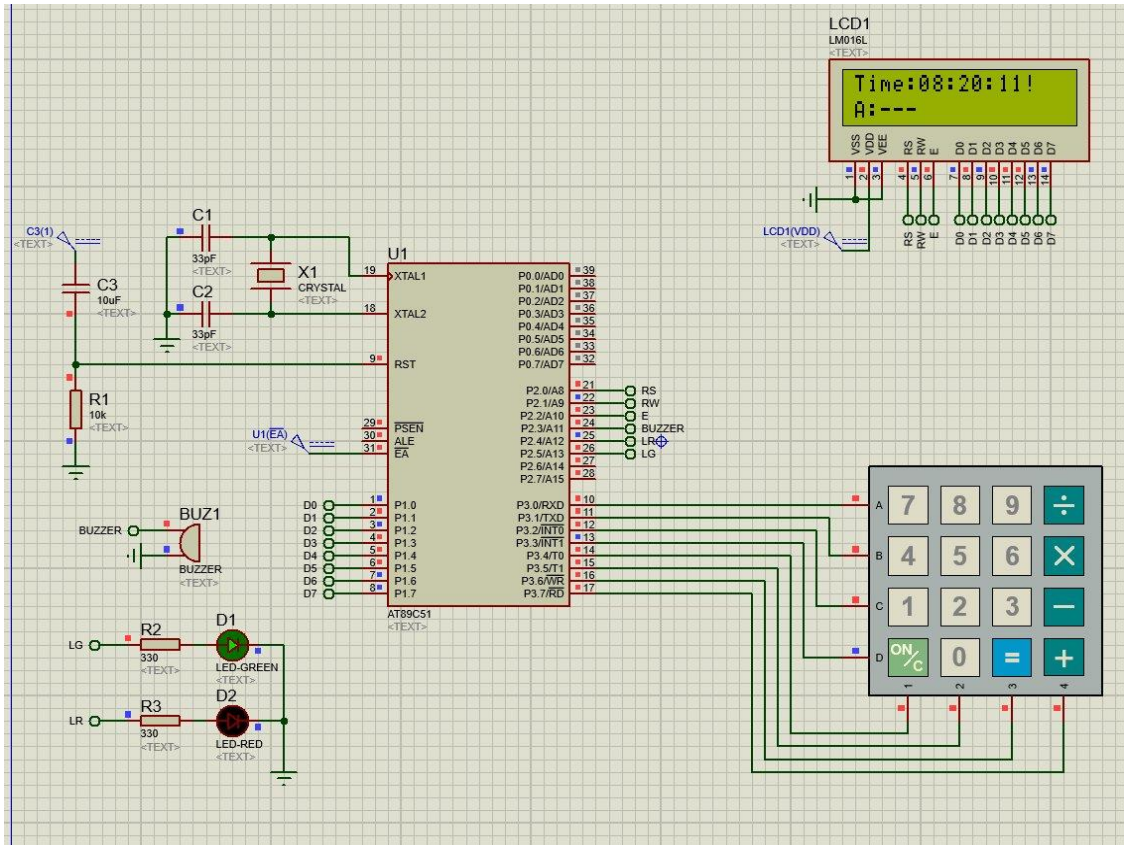

Figure 1: Proteus schematic of the 8051-based digital alarm clock.

5.1 Microcontroller and Clock Circuit

The AT89C51 (U1) serves as the central controller. Pins XTAL1 and XTAL2 are connected to a 12 MHz quartz crystal (X1) with two 33 pF ceramic capacitors (C1, C2) referenced to ground, forming the Pierce oscillator that generates the internal system clock. At 12 MHz, one machine cycle equals 1 μs, which simplifies time-based calculations throughout the firmware.

5.2 Reset Circuit

A standard RC power-on reset circuit is implemented using C3 (10 μF) in series with R1 (10 kΩ). The capacitor holds pin 9 (RST) high for several milliseconds after power-up, allowing the crystal oscillator to stabilise before the CPU begins executing code. Because the EA pin (pin 31) is tied to V_{ee}, execution proceeds from the internal Flash program memory.

5.3 LCD Display Interface

A 16×2 character LM016L LCD module is used as the primary output device. The eight data lines (D0-D7) are connected to Port 1 (P1.0 - P1.7), while the three control lines (RS, R/W, E) are connected to P2.0, P2.1, and P2.2 respectively. The LCD is initialised in 8-bit mode, two-line, 5×8 dot-matrix configuration, with the cursor disabled and the display enabled (0x0C).

5.4 Keypad Interface

A 4×4 matrix keypad provides user input. The four rows (labelled A-D) are connected to P3.0–P3.3, and the four columns (labelled 1-4) are connected to P3.4–P3.7. The firmware performs standard row-scan polling: each row is driven low in turn, and the column lines are read to identify a pressed key. Weak internal pull-ups on Port 3 ensure that idle column lines read as logic high.

5.5 Buzzer and LED Indicators

The piezoelectric buzzer (BUZ1) is driven directly from P2.3 in an active-low configuration: setting the pin low energises the buzzer, while driving it high silences it. Two LEDs (D1 green and D2 red) are connected to P2.5 and P2.4 respectively, each through a 330 Ω current-limiting resistor (R2, R3) that sets the forward current to approximately 10 mA. The LEDs provide visual status indication: LED1 flashes in synchronism with the buzzer during an active alarm, while LED2 indicates system ready status.

6. Bill of Materials (BOM)

Table 1 presents the complete bill of materials for the digital alarm clock prototype. Prices indicated are approximate retail costs observed in the local electronics market (Dhaka, Bangladesh), rounded for readability. Generic equivalents were used wherever specific part numbers were not critical, which keeps the overall build affordable while preserving functional integrity.

Sl.	Component	Designator	Qty	Value / Specification	Unit Cost (BDT)
1	Microcontroller	U1	1	AT89C51, 40-pin DIP	160
2	16×2 LCD Module	LCD1	1	LM016L, HD44780-compatible	220
3	4×4 Matrix Keypad	…	1	Membrane, 16 keys	90
4	Crystal Oscillator	X1	1	12 MHz quartz, HC-49S	18
5	Ceramic Capacitor	C1, C2	2	33 pF, 50 V	2
6	Electrolytic Capacitor	C3	1	10 μF, 16 V	3
7	Resistor	R1	1	10 kΩ, 1/4 W, 5%	1
8	Resistor	R2, R3	2	330 Ω, 1/4 W, 5%	1
9	LED	D1	1	Green, 5 mm, diffused	3
10	LED	D2	1	Red, 5 mm, diffused	3
11	Piezo Buzzer	BUZ1	1	5 V, active-low, 85 dB	25
12	IC Socket	…	1	40-pin DIP machined	20
13	Perfboard / Zero-PCB	…	1	10 cm × 15 cm, FR-4	80
14	Connecting Wires	…	1	Single-core, assorted (lot)	40
15	5 V Power Supply	…	1	7805 regulator or USB adapter	60
16	Pin Headers	…	1	Male, 2.54 mm (strip)	25
				Total Estimated Cost:	≈ 754 BDT

Table 1: Bill of Materials for the Digital Alarm Clock Prototype

The total material cost of approximately 754 BDT (roughly equivalent to 6–7 $ at current exchange rates) confirms that the design meets its "low-cost" objective. This cost could be further reduced in a production context through bulk procurement, migration to an SMD-based PCB, and potential consolidation of the reset and supply circuitry.

7. Software Design and Firmware Overview

The firmware was developed in 8051 assembly language using the MIDE-51 assembler. Assembly was chosen over a high-level language (such as C) for two reasons: first, it allowed tight control over timing-critical sections such as the timer interrupt service routine and the LCD driver; second, it served the pedagogical goal of reinforcing the principles of 8051 architecture and instruction-set programming.

7.1 RAM Memory Map

All mutable state is kept in the internal 128-byte RAM of the AT89C51, starting at address 30H to avoid collision with the register banks and the stack. Table 2 summarises the memory allocation.

Address	Label	Purpose
30H–32H	SEC, MINR, HOUR	Current time registers (binary, not BCD)
33H–35H	A1HR, A1MN, A1EN	Alarm 1 hour, minute, and enable flag
36H–38H	A2HR, A2MN, A2EN	Alarm 2 hour, minute, and enable flag
39H–3BH	A3HR, A3MN, A3EN	Alarm 3 hour, minute, and enable flag
3CH	TICK	100 Hz tick counter (0–99, rolls over once per second)
3DH–40H	SNZMIN, SNZACT, SNZHR, SNZMN	Snooze duration, active flag, target time
41H	SEQIDX	Current index in the 1→2→1 stop sequence
42H–44H	DC1, DC2, DC3	Independent counters for nested delay loops
50H–53H	V1, V2, V3, VN	General-purpose working variables

Table 2: Internal RAM Memory Map

7.2 Timer 0 Interrupt Service Routine (ISR)

Accurate timekeeping is the cornerstone of the design. Timer 0 is operated in Mode 1 (16-bit) with a reload value of 0xD8F0 (equivalent to 55,536), which yields an overflow period of exactly 10 000 machine cycles or 10 ms at a 12 MHz crystal frequency. On each interrupt, the ISR reloads the timer, increments the TICK variable, and cascades into SEC, MINR, and HOUR as the appropriate counters roll over. The ISR is kept deliberately short (pushing only ACC and PSW onto the stack) to minimise interrupt latency and preserve main-loop responsiveness.

7.3 LCD Driver

The LCD driver follows the standard HD44780 initialisation sequence: three successive 0x30 commands to force 8-bit mode, followed by function set (0x38), display off (0x08), clear (0x01), entry mode (0x06), and finally display on (0x0C). The primary driver routines are LCD_CMD (write command), LCD_WDAT (write data), and LCD_PSTR (print null-terminated string from code memory). A helper routine, LCD_CH,

combines a clear-display and cursor-home into a single call, saving redundant LCALLs throughout the firmware.

7.4 Keypad Scanner

Two keypad access routines are provided: GET_KEY (blocking), which waits indefinitely for a key press and returns its ASCII code in V1; and CHK_KEY_NB (non-blocking), used inside the main loop to poll without halting time display updates. Both routines use row-scan multiplexing with a 20 ms software debounce and a shared KEY_RELEASE routine that waits for all keys to be released before returning. This prevents a single physical press from being registered multiple times.

7.5 Alarm and Snooze Logic

On each iteration of the main loop, the CHK_ALARMS routine compares the current HOUR, MINR, and SEC registers against each enabled alarm. The SEC = 0 condition ensures that each alarm triggers exactly once per minute, preventing spurious repeats. When an alarm fires, its enable flag is cleared (one-shot semantics), and ALM_ROUTINE is invoked. This routine alternates the buzzer and LED1 on and off in 300 ms windows to produce the characteristic beeping pattern, while concurrently polling the keypad for a stop or snooze command. The snooze function prompts the user for a delay (1 - 59 minutes) and re-arms the alarm at the corresponding future time, with correct handling of hour and day rollover.

7.6 1→2→1 Stop Sequence

To discourage accidental alarm dismissal, the design requires the user to enter the three-key sequence 1→2→1 before a non-snooze stop is accepted. The CHK_SEQ routine maintains an index (SEQIDX) through this pattern. If the user presses a wrong key, the sequence resets; unless the wrong key happens to be "1", in which case the index rewinds to 1 rather than 0, giving the user a smart recovery path. The key "D" acts as an emergency immediate stop and bypasses the sequence, while the key "A" activates snooze.

8. Working Flowchart

Figure 2 presents the working flowchart of the entire firmware, covering both the main loop and the alarm sub-routine. The main loop continuously refreshes the time display, scans for active alarms, and polls the keypad; a non-zero key press transfers control to the corresponding menu handler. When an alarm triggers, the flow transfers to the ALM_ROUTINE sub-flow shown in the lower half of the figure, returning to the main loop only after the user has successfully stopped or snoozed the alarm.

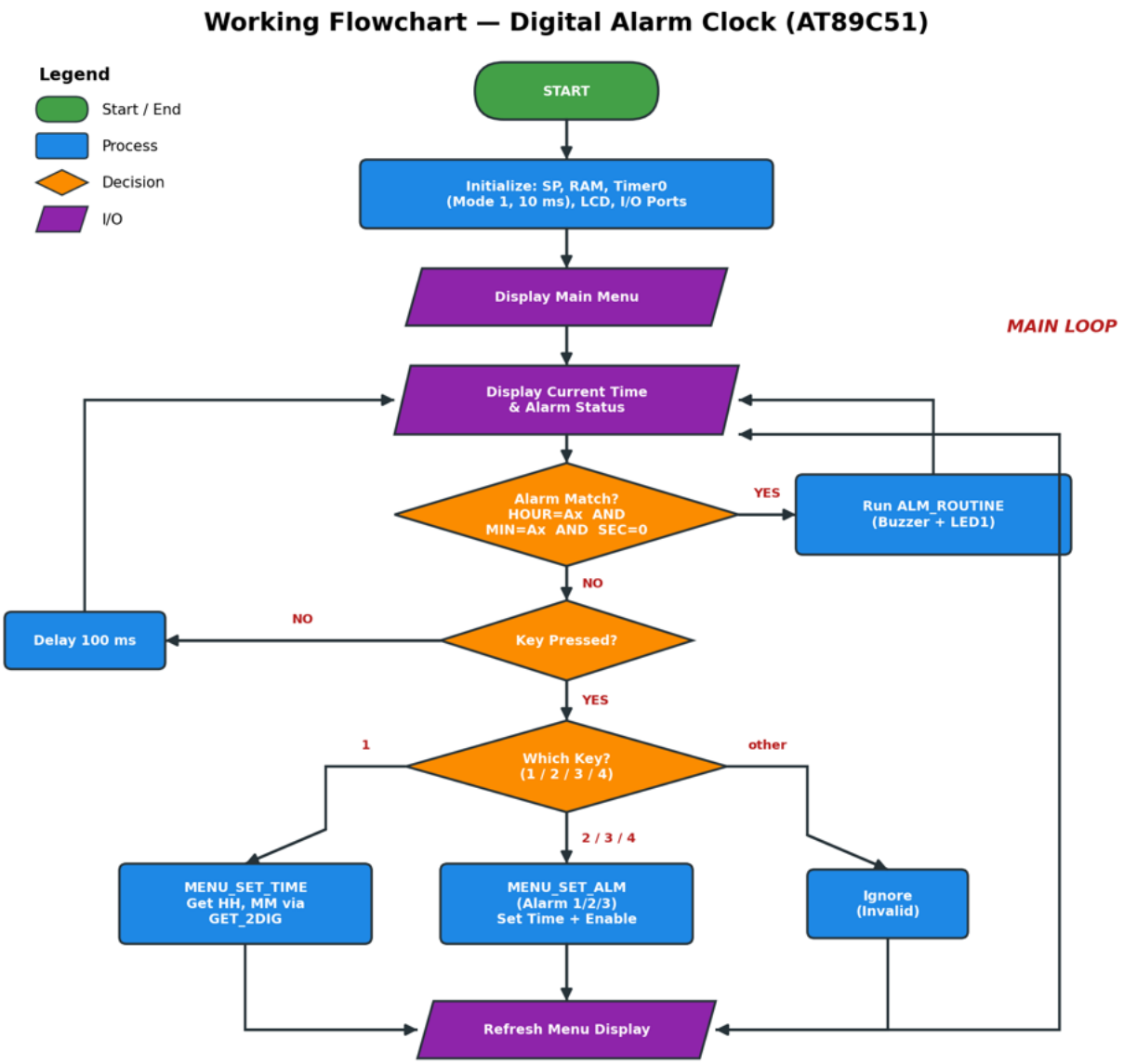

Figure 2.1: Complete firmware flowchart (main loop).

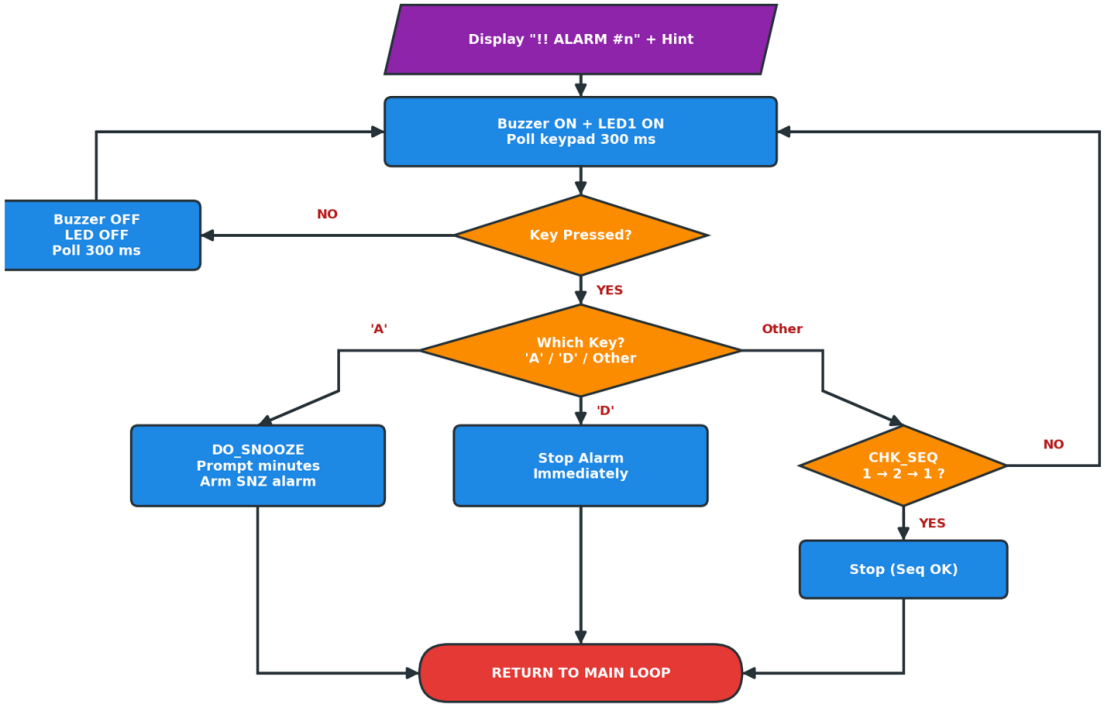

Figure 2.2: Complete firmware flowchart (ALM routine sub-flow).

9. Operational Description and User Interface

From the user's point of view, the device presents a simple two-line menu at all times when no alarm is active. Line 1 of the LCD displays the current time in HH:MM:SS format along with a compact alarm-status indicator ("A:1--" for example, meaning Alarm 1 is armed and Alarms 2/3 are disarmed). Line 2 shows a condensed menu of the four available actions, listed below.

9.1 Main Menu Options

- Key 1: Set current time (prompts for hour then minute).
- Key 2: Configure Alarm 1 (prompts for hour and minute, then arms the alarm).
- Key 3: Configure Alarm 2.
- Key 4: Configure Alarm 3.

9.2 Alarm Active Behaviour

When any armed alarm matches the current time (at the zero-second boundary), the LCD displays "!! ALARM #n" on the first line and the hint "1>2>1 A=Snz D=X" on the second. The buzzer and LED1 alternate on and off in 300 ms windows. The user has three ways to respond:

- Press "A" to activate snooze. The device then prompts for a delay of between 1 and 59 minutes and rearms internally for that offset.
- Press "D" for an immediate emergency stop. This bypasses the multi-key sequence and is intended for situations where the user must silence the alarm quickly (for example, if testing the circuit).
- Enter the three-key sequence 1→2→1 to perform a normal stop. The device confirms with the message "Seq Correct :)" before returning to the main menu.

9.3 Time and Alarm Setting

Whenever the firmware prompts for a numeric value (hour, minute, or snooze duration), it reads two successive digits via the GET_2DIG subroutine. Each digit is echoed on the LCD as it is pressed, providing immediate feedback. The two digits are then combined into a binary value that is stored directly in the relevant RAM location. After a successful setting, a confirmation message is displayed for approximately one second before returning to the main menu.

10. Code flow:

1. Reset Vector & Boot

The chip resets at 0x0000 and jumps immediately to MAIN. Timer 0 interrupt is wired to 0x000B.

Reset / Vector Table (ORG 0000H – 000BH)	
0x0000 LJMP MAIN	Unconditional jump to application entry point.
0x000B LJMP T0_ISR	Timer 0 overflow vector → 10 ms tick ISR.

2. T0_ISR — Timer 0 Interrupt Service Routine

Location: ORG 0100H | Triggered every 10 ms by Timer 0 overflow (reload = 0xD8F0 @ 12 MHz)

T0_ISR Flow	
PUSH ACC / PSW	Save accumulator and program status word.
Reload TH0/TL0	Write 0xD8F0 to maintain 10 ms period.
INC TICK	Increment 10 ms tick counter (0–99).
TICK == 100?	If yes: clear TICK, increment SEC.
SEC == 60?	If yes: clear SEC, increment MINR.
MINR == 60?	If yes: clear MINR, increment HOUR.
HOUR == 24?	If yes: clear HOUR (midnight wrap).
POP PSW / ACC	Restore context.
RETI	Return from interrupt.

3. MAIN — Initialisation & Main Loop

Location: ORG 0200H | Entry point after reset.

MAIN — Initialisation	
SP = 0x6F	Set stack pointer above register banks.
Clear time vars	SEC, MINR, HOUR initialised to 00.
Clear alarm enables	A1EN, A2EN, A3EN = 00 (all alarms off).
SNZMIN = 05H	Default snooze duration = 5 minutes.
SNZACT, SEQIDX = 0	Snooze inactive; sequence checker reset.
BUZZER = 1 (off)	Active-LOW buzzer; high = silent.
LEDs init	LED1 off, LED2 on.
TMOD = 01H	Timer 0 in 16-bit mode.
Load TH0/TL0	0xD8F0 for first 10 ms period.
SETB ET0, EA, TR0	Enable T0 interrupt, global interrupt; start timer.
LCALL LCD_INIT	Initialise HD44780 LCD (8-bit mode, 2-line).
LCALL SHOW_MENU	Display main menu on LCD.

MAIN LOOP (MLOOP — infinite)	
LCALL DISP_TIME	Refresh LCD line 1 (HH:MM:SS) and alarm-enable indicators.
LCALL CHK_ALARMS	Check all three alarms + snooze against current time.
LCALL CHK_KEY_NB	Non-blocking key scan; result in V1 (0 = no key).
V1 == '1'?	Yes → LCALL MENU_SET_TIME, then SHOW_MENU.
V1 == '2'?	Yes → VN=1, LCALL MENU_SET_ALM, then SHOW_MENU.
V1 == '3'?	Yes → VN=2, LCALL MENU_SET_ALM, then SHOW_MENU.
V1 == '4'?	Yes → VN=3, LCALL MENU_SET_ALM, then SHOW_MENU.
LCALL DLY_100MS	100 ms loop delay, then LJMP MLOOP.

4. CHK_ALARMS — Alarm & Snooze Checker

Called once per main loop iteration. Checks snooze first, then Alarms 1–3 in order. Disables an alarm after it fires.

CHK_ALARMS Flow	
SNZACT != 0?	Yes: compare HOUR/MINR/SEC==0 with SNZHR/SNZMN. Match → VN=0, LCALL ALM_ROUTINE.
A1EN != 0?	Yes: compare HOUR/MINR/SEC==0 with A1HR/A1MN. Match → A1EN=0, VN=1, LCALL ALM_ROUTINE.
A2EN != 0?	Yes: compare with A2HR/A2MN. Match → A2EN=0, VN=2, LCALL ALM_ROUTINE.
A3EN != 0?	Yes: compare with A3HR/A3MN. Match → A3EN=0, VN=3, LCALL ALM_ROUTINE.
RET	Return to main loop.

5. ALM_ROUTINE — Alarm Handler

Drives the buzzer + LED1 in an on/off cycle while polling the keypad. Accepts three stop methods:

- Key 'D' — Immediate stop.
- Key 'A' — Snooze (calls DO_SNOOZE).
- Sequence 1→2→1 — Stop via CHK_SEQ (Extra Feature).

ALM_ROUTINE Flow (SEQIDX reset to 0 on entry)	
ALML (loop top)	LCD_CH + print 'ALARM n' on line 1, hint text on line 2.
CLR BUZZER / SETB LED1	Buzzer ON (active-LOW), LED1 ON.
LCALL ALM_POLL	Poll keypad for 300 ms. Key stored in V1.
V1 != 0?	Key detected → jump to AGOT.
SETB BUZZER / CLR LED1	Buzzer OFF, LED1 OFF (silent phase).
LCALL ALM_POLL	Poll keypad for another 300 ms.
V1 == 0?	Still no key → loop back to ALML.
AGOT: key in A	Check key value.
A == 'A'?	Yes → LCALL DO_SNOOZE, RET.
A == 'D'?	Yes → print 'Alarm Stopped!', DLY_1S, RET.
else: LCALL CHK_SEQ	Validate sequence progress; V3=1 if complete.
V3 == 0?	Sequence incomplete → loop to ALML.
Sequence OK	Print 'Alarm Stopped! / Seq Correct :)', DLY_1S, reset SEQIDX, RET.

6. ALM_POLL — 300 ms Keypad Poll

Scans for a key press in thirty 10 ms slices. Returns immediately if a key is found.

ALM_POLL Flow	
DC3 = 30, V1 = 0	Initialise loop counter (30 × 10 ms = 300 ms) and clear result.
LCALL DLY_10MS	Wait 10 ms.
LCALL CHK_KEY_NB	Non-blocking key scan → V1.
V1 != 0?	Key pressed → exit immediately (APDN).
DJNZ DC3, APLP	Decrement counter; repeat if not zero.
APDN: RET	Return (V1 = key or 0x00).

7. CHK_SEQ — Sequence Validator (1 → 2 → 1)

Maintains a three-state machine in SEQIDX to detect the unlock sequence. Returns V3=1 when the full 1→2→1 sequence is completed.

CHK_SEQ State Machine	
State 0 (SEQIDX=0)	Expect '1'. Match → SEQIDX=1. Wrong key → reset (or stay at 1 if key='1').
State 1 (SEQIDX=1)	Expect '2'. Match → SEQIDX=2. Wrong key → reset (restart at 1 if key='1').
State 2 (SEQIDX=2)	Expect '1'. Match → V3=1 (sequence complete), SEQIDX=0. Wrong key → reset.
V3 = 0 (default)	Sequence not yet complete; caller loops back.

8. DO_SNOOZE — Snooze Setup

Prompts user for a duration (1–59 min), computes the target wake time with 24-hour rollover, and arms the snooze alarm.

DO_SNOOZE Flow	
KEY_RELEASE	Wait for all keys to be released.
Buzzer OFF, LED1 OFF	Silence buzzer; clear LED.
LCD prompt	'Snooze Duration / Mins(01-59):' on LCD.
LCALL GET_2DIG	Read two-digit duration into V1 → SNZMIN.
Compute SNZMN	SNZMN = MINR + SNZMIN.
SNZHR = HOUR	Copy current hour as base.
Rollover loop	While SNZMN >= 60: SNZMN -= 60, INC SNZHR. If SNZHR == 24: SNZHR = 0.
SNZACT = 1	Arm the snooze flag (checked by CHK_ALARMS).
Print 'Snoozed!'	Confirm on LCD for 1 s, then RET.

9. MENU_SET_TIME — Set Current Time

Interactive two-digit entry for hour and minute. Resets seconds to 00 on confirmation.

MENU_SET_TIME Flow	
KEY_RELEASE	Flush any held key.
LCD_CH + header	Clear LCD, print '--- Set Time ---'.
Prompt 'Hour(00-23)'	Line 2 prompt.
GET_2DIG → HOUR	Read 2-digit hour; store in HOUR.
Prompt 'Min(00-59)'	Line 2 prompt.
GET_2DIG → MINR	Read 2-digit minute; store in MINR.
SEC = 00	Reset seconds.
Print 'Time Set OK!'	Confirm for 1 s, RET.

10. MENU_SET_ALM — Set Alarm (VN = 1, 2 or 3)

Sets hour and minute for the alarm number passed in VN, then enables it. Uses branching on VN to write to the correct RAM variables.

MENU_SET_ALM Flow	
KEY_RELEASE	Flush any held key.
LCD: 'Set Alarm #n'	Print alarm number from VN.
Prompt Hour	Line 2: 'Hour(00-23)'.
GET_2DIG	Read hour digit pair.
Store to A1HR/A2HR/A3HR	Branch on VN (1/2/3) to write correct register.
Prompt Minute	Line 2: 'Min(00-59)'.
GET_2DIG	Read minute digit pair.
Store to A1MN…A3MN	Write minute, then set corresponding AxEN = 01H.
Print 'Alarm n Set OK!'	Confirm on LCD for 1 s, RET.

11. DISP_TIME — Refresh Time Display

Updates both LCD lines every main loop cycle. Line 1 shows HH:MM:SS; line 2 shows which alarms are active.

DISP_TIME Flow	
LCD_HOME	Move cursor to row 0, col 0.
Print 'Time:'	Static label.
P2D(HOUR)	Print hours as two decimal digits.
Print ':'	Separator.
P2D(MINR)	Print minutes.
Print ':'	Separator.
P2D(SEC)	Print seconds.
LCD_LINE2	Move cursor to row 1.
Print 'A:' + status	For each of A1EN/A2EN/A3EN: print digit (1/2/3) if enabled, '-' if disabled.
Trailing spaces	Clear any stale characters. RET.

12. SHOW_MENU — Display Main Menu

Clears LCD and prints a static two-line menu. Called once after init and after each menu operation completes.

SHOW_MENU Flow	
LCD_CH	Clear + home cursor.
Line 1	Print '1:Time 2:Alm1'.
LCD_LINE2	Move to row 1.
Line 2	Print '3:Alm2 4:Alm3'. RET.

13. P2D — Print Byte as 2 Decimal Digits

Divides A by 10; sends tens digit then units digit to LCD_WDAT.

P2D Flow (A = binary value 0–99)	
DIV AB (B=10)	A = tens quotient; B = units remainder.
A + 0x30 → LCD	Convert tens to ASCII, call LCD_WDAT.
B + 0x30 → LCD	Convert units to ASCII, call LCD_WDAT.
RET	Returns; A and B modified.

14. GET_2DIG — Read Two-Digit Decimal Input

Blocks until two keys are pressed. Echoes each digit to LCD and converts the ASCII pair to a binary value in V1.

GET_2DIG Flow	
GET_KEY → V1	Block until first keypress (tens digit); echo to LCD.
V1 - 0x30 → V2	Strip ASCII offset; save tens digit in V2.
GET_KEY → V1	Block until second keypress (units digit); echo to LCD.
V1 - 0x30 → V1	Strip ASCII offset; units in V1.
V1 = V2×10 + V1	MUL AB with V2, add units → binary result in V1.
RET	V1 holds binary value (0–99).

15. KEY_RELEASE — Wait for All Keys Released

Drives all column lines low, reads P3 rows, and loops until all row bits read high (no key pressed). Adds 20 ms debounce delay before returning.

KEY_RELEASE Flow	
CLR KR1–KR4	Pull all columns low to sense any row.
Read P3 & 0xF0	Mask column bits; check row nibble.
!= 0xF0?	Some key still held → loop.
LCALL DLY_20MS	Debounce delay.
RET	All keys confirmed released.

16. GET_KEY — Blocking Keypad Scan

Scans all four rows by driving each column low in turn. Debounces with 20 ms delay, identifies the column, and maps row/column to an ASCII character stored in V1. Calls KEY_RELEASE before returning.

GET_KEY Flow	
GKLP: drive col 1 low	Activate KR1; read P3.
Row hit? → GR1	Yes: DLY_20MS, read column nibble, map to '1'/'2'/'3'/'A'.
Drive col 2 low	Activate KR2; read P3.
Row hit? → GR2	Yes: map to '4'/'5'/'6'/'B'.
Drive col 3 low	Activate KR3; read P3.
Row hit? → GR3	Yes: map to '7'/'8'/'9'/'C'.
Drive col 4 low	Activate KR4; read P3.
Row hit? → GR4	Yes: map to '*'/'0'/'#'/'D'.
No hit → LJMP GKLP	No key found; scan again.
KRET: KEY_RELEASE	Shared exit: wait for release, RET.

17. CHK_KEY_NB — Non-Blocking Key Check

Same column-scan logic as GET_KEY but returns immediately with V1=0 if no key is found. If a key is detected it tail-calls GET_KEY (LJMP) so GET_KEY does the debounce and decode.

CHK_KEY_NB Flow	
V1 = 0	Clear result before scanning.
Scan 4 columns	Same row-drive/read sequence as GET_KEY.
Any row low?	Yes → LJMP GET_KEY (tail-call; GET_KEY RET returns to our caller).
No key	All rows high → RET with V1=0.

18. LCD Driver — HD44780 Interface Routines

Eight-bit parallel interface via P1 (data) and P2.0–P2.2 (RS, RW, EN). All routines call delay helpers to meet LCD timing.

LCD Subroutine Summary	
LCD_INIT	Power-on sequence: three 0x30 resets, then function-set (0x38), display-off, clear (0x01), entry-mode (0x06), display-on (0x0C).
LCD_CMD	Write command byte: P1=A, RS=0, RW=0, EN pulse with 1 ms settle.
LCD_WDAT	Write data byte: P1=A, RS=1, RW=0, EN pulse with 1 ms settle.
LCD_CH	Combined clear (0x01) + 5 ms delay + home (0x80). Saves one LCALL vs calling clear then home separately.
LCD_HOME	Send 0x80 command → cursor row 0, col 0.
LCD_LINE2	Send 0xC0 command → cursor row 1, col 0.
LCD_PSTR	Print null-terminated string from DPTR in code memory (MOVC loop until 0x00).
LCD_WDAT_V1	Helper: move V1 → A, call LCD_WDAT, restore A=V1.

19. Delay Routines

Hierarchical delay stack using three dedicated counters: DC1 (innermost), DC2 (mid-level), DC3 (outer-level). All calibrated for 12 MHz (1 machine cycle = 1 μs).

Delay Subroutine Hierarchy	
DLY_1MS	Two 250-iteration DJNZ loops (DC1=0xFA each) → ≈ 1 ms. Base primitive.
DLY_2MS	DC2=2, calls DLY_1MS twice.
DLY_5MS	DC2=5, calls DLY_1MS five times.
DLY_10MS	DC2=10, calls DLY_1MS ten times.
DLY_20MS	DC2=20, calls DLY_1MS twenty times. Used for key debounce.
DLY_100MS	DC3=20, calls DLY_5MS twenty times → 100 ms. Used in main loop.
DLY_1S	DC3=200, calls DLY_5MS 200 times → ≈ 1 s. Used for LCD confirmations.

20. Call Graph Summary

Top-down call relationships. Arrows show caller → callee.

Call Graph	
MAIN	→ LCD_INIT, SHOW_MENU, DISP_TIME, CHK_ALARMS, CHK_KEY_NB, MENU_SET_TIME, MENU_SET_ALM, DLY_100MS
CHK_ALARMS	→ ALM_ROUTINE
ALM_ROUTINE	→ LCD_CH, LCD_PSTR, LCD_WDAT, ALM_POLL, DO_SNOOZE, CHK_SEQ, DLY_1S
ALM_POLL	→ DLY_10MS, CHK_KEY_NB
CHK_KEY_NB	→ GET_KEY (tail-call)
GET_KEY	→ DLY_20MS, KEY_RELEASE
DO_SNOOZE	→ KEY_RELEASE, LCD_CH, LCD_PSTR, GET_2DIG, DLY_1S
MENU_SET_TIME	→ KEY_RELEASE, LCD_CH, LCD_PSTR, LCD_LINE2, GET_2DIG, DLY_1S
MENU_SET_ALM	→ KEY_RELEASE, LCD_CH, LCD_PSTR, LCD_LINE2, LCD_WDAT, GET_2DIG, DLY_1S
DISP_TIME	→ LCD_HOME, LCD_PSTR, P2D, LCD_WDAT, LCD_LINE2
GET_2DIG	→ GET_KEY, LCD_WDAT_V1
LCD_INIT / LCD_CMD / LCD_WDAT	→ DLY_1MS, DLY_2MS, DLY_5MS, DLY_20MS
DLY_2/5/10/20MS	→ DLY_1MS
DLY_100MS / DLY_1S	→ DLY_5MS → DLY_1MS

11. Testing and Results

The complete system was verified in the Proteus VSM environment by executing the assembled HEX file on a simulated AT89C51 and interacting with the virtual LCD, keypad, buzzer, and LEDs. Table 3 summarises the twelve principal test cases that were executed, together with their observed outcomes. All cases produced the expected behaviour, validating both the firmware and the hardware design at the schematic level.

Test ID	Test Case Description	Observed Result	Status
T1	Power-on reset initialises time to 00:00:00	Time starts at 00:00:00	PASS
T2	Timer 0 ISR increments SEC every 1 second	Seconds advance correctly in simulation	PASS
T3	Minute/hour rollover at 59:59 → next hour	HH:MM rolls over as expected	PASS
T4	Hour rollover at 23:59:59 → 00:00:00	Rolls to 00:00:00 cleanly	PASS
T5	Set time via Key 1 and two 2-digit entries	Time updates to user-entered value	PASS
T6	Arm Alarm 1 and verify trigger at target time	Alarm triggers exactly at HH:MM:00	PASS
T7	LED1 flashes synchronously with buzzer	300 ms on / 300 ms off pattern confirmed	PASS
T8	Stop alarm using 1→2→1 sequence	"Seq Correct :)" displayed, alarm stops	PASS
T9	Emergency stop using "D"	Alarm stops immediately, "Alarm Stopped!" shown	PASS
T10	Snooze for 5 minutes, verify re-trigger	Alarm fires again after 5 minutes	PASS
T11	Three simultaneous alarms at different times	Each fires at its own configured time	PASS
T12	Keypad debouncing under rapid presses	No duplicate registrations observed	PASS

Table 3: Test Cases and Observed Results

During extended simulation runs, the Timer 0 based time-keeping was observed to agree with the simulator's reference clock to within the resolution of the simulation engine. On physical hardware, however, a small amount of drift is expected due to crystal tolerance (typically ±20 to 50 ppm), corresponding to a few seconds of error per day. This limitation is addressed in Section 12.

12. Limitations

While the current prototype meets all of its stated functional objectives, a candid review of the implementation reveals several limitations that would need to be addressed before the design could be considered production-ready:

- **Time drift under power loss:** The device currently derives time from Timer 0 counted against the 12 MHz crystal. A power interruption, however brief, resets the clock to 00:00:00 and erases all configured alarms. There is no battery-backed real-time clock and no non-volatile storage.
- **Crystal tolerance:** Standard 12 MHz quartz crystals exhibit ±20 to 50 ppm tolerance, which translates to a few seconds of drift per day. Over weeks, this accumulates into noticeable error.
- **Limited input validation:** The GET_2DIG routine accepts any two key presses and subtracts 0x30 to convert ASCII to binary. If the user presses a non-digit key such as "A", the resulting value will be nonsensical (for instance, 17 rather than an error). Robust input validation is required for deployment.
- **No 12-hour (AM/PM) display mode:** The device operates exclusively in 24-hour format. Many users in regions where 12-hour conventions dominate may find this inconvenient.
- **Fixed alarm tone and volume:** The buzzer produces a single, unvarying tone at a fixed duty cycle. There is no provision for melodies, adjustable volume, or gradually-increasing intensity.
- **No calendar or date functionality:** The design tracks only hours, minutes, and seconds. There is no day-of-week, date, or month information, which precludes recurring weekly alarms.
- **LCD driver inefficiency:** The current driver uses fixed millisecond-scale delays between every write instead of polling the busy flag. This slows down display refresh and contributes to minor visible lag in the time display.

13. Future Improvement Prospects

The current design provides a solid foundation on which a number of meaningful enhancements could be built. The suggested improvements are grouped below by theme.

13.1 Hardware Enhancements

- **Dedicated Real-Time Clock (RTC):** Interfacing a DS1307 or DS3231 RTC chip via the I²C bus (using two P2 pins) would provide crystal-accurate timekeeping independent of the microcontroller, as well as full calendar information (day, date, month, year) and battery-backed retention across power loss.
- **Battery backup:** Adding a CR2032 coin cell in conjunction with the RTC chip would preserve time settings during power outages, a standard feature of commercial alarm clocks.
- **EEPROM for alarm persistence:** A 24C02-series I²C EEPROM would allow alarm settings to survive power cycles, eliminating the need to reconfigure alarms on every power-up.

- **Audio improvements:** Replacing the simple buzzer with a DFPlayer Mini module would allow playback of user-selected MP3 melodies. A transistor-driven speaker and digital-to-analogue conversion (via PWM) could produce varied tones.
- **Ambient light-dependent brightness:** An LDR connected to an ADC channel (via an external ADC such as the ADC0808) could automatically dim the LCD backlight and LED indicators at night, improving bedside usability.

13.2 Firmware Enhancements

- **Input validation:** Modifying GET_2DIG to reject non-digit characters and out-of-range values would prevent corrupted time/alarm settings.
- **12/24-hour display mode:** Adding a user-selectable preference (stored in EEPROM) for AM/PM versus 24-hour format would significantly improve accessibility.
- **Recurring alarms:** With RTC support providing day-of-week information, the firmware could support alarms that repeat on selected weekdays (for example, weekdays-only wake-up).
- **Gradual volume ramp:** Replacing the binary buzzer output with PWM would allow a gradual increase in volume over the first 30 seconds of an alarm, producing a gentler wake-up experience.
- **Smart snooze scaling:** The snooze duration could decrease on each successive press (for example, 10 minutes, then 7, then 5), discouraging chronic over-snoozing.
- **LCD busy-flag polling:** Replacing the conservative delay-based LCD timing with active busy-flag polling would reduce display write time by roughly an order of magnitude, eliminating visible lag.

13.3 User Interface Enhancements

- **Encoder-based input:** Replacing or augmenting the keypad with a rotary encoder would speed up numeric entry and feel more natural for time-setting tasks.
- **Graphical OLED display:** Upgrading from the character LCD to a 128×64 SSD1306 OLED (via I^2C) would enable large, easy-to-read clock numerals and visual alarm icons.
- **Wireless connectivity:** A more ambitious upgrade would involve migrating the design to an ESP32 and adding Wi-Fi NTP synchronisation, smartphone app control, and cloud-based alarm management.
- **Enclosure design:** A custom 3D-printed enclosure with tactile buttons and a recessed display would transform the prototype into a presentable finished product.

14. Conclusion

This project has successfully achieved its primary goal of designing and implementing a low-cost, feature-rich digital alarm clock on the AT89C51 microcontroller platform. The completed system demonstrates accurate interrupt-driven timekeeping, a responsive user interface, three independent alarms with configurable snooze, and a novel multi-key stop-sequence mechanism that adds genuine usability value over conventional single-button designs.

Beyond meeting the functional specifications, the project provided valuable hands-on exposure to the full stack of embedded-systems design: from schematic capture in Proteus, through 8051 assembly programming in MIDE-51, to end-to-end system verification. The iterative development process; culminating in the present optimised firmware version; reinforced the importance of modular code organisation, careful attention to interrupt latency, robust debouncing, and defensive input handling.

The total material cost of approximately 754 BDT validates the design's low-cost objective, and the identified limitations provide a clear roadmap for future enhancement. With modest upgrades such as the addition of a DS3231 RTC, non-volatile alarm storage, and a custom enclosure, the prototype could readily evolve into a polished consumer product. More ambitiously, migration to a wireless platform such as the ESP32 would open up network-synchronised timekeeping and smartphone integration.

Overall, the project serves both as a functional deliverable and as a comprehensive pedagogical vehicle, exemplifying how a classical 8-bit microcontroller can still deliver meaningful, well-engineered solutions when paired with careful design and thoughtful firmware.

15. References

- [1] Atmel Corporation, *AT89C51 8-bit Microcontroller with 4K Bytes Flash Datasheet*, Document 0265H–MICRO–10/02, 2002.
- [2] M. A. Mazidi, J. G. Mazidi, and R. D. McKinlay, *The 8051 Microcontroller and Embedded Systems Using Assembly and C*, 2nd ed. Pearson Education, 2006.
- [3] Hitachi Ltd., *HD44780U (LCD-II) Dot Matrix Liquid Crystal Display Controller/Driver Datasheet*, ADE-207-272(Z), 1998.
- [4] W. W. W. ASEM-51, *ASEM-51 User's Manual*, W. W. Heinz, 2002.
- [5] Labcenter Electronics, *Proteus VSM Simulator User Guide*, Labcenter Electronics Ltd., 2023.
- [6] K. J. Ayala, *The 8051 Microcontroller: Architecture, Programming and Applications*, 3rd ed. Cengage Learning, 2004.
- [7] Maxim Integrated, *DS3231 Extremely Accurate I²C-Integrated RTC/TCXO/Crystal Datasheet*, Rev. 10, 2015.